# P.I.T.C.H. Midterm Performance Evaluation Report

*March 15, 2026*


## Abstract

This midterm report presents the current performance evaluation progress for P.I.T.C.H., a microservice-based AI-assisted web application. The current dataset covers baseline characterization and light API load-scaling experiments at 10, 50, 100, 200, and 500 concurrent virtual users using k6. Initial results show that the light endpoint performs well at low load, but tail latency degrades sharply by 50 and 100 virtual users. At 200 virtual users the system becomes unstable, and at 500 virtual users the gateway enters overload behavior with partial and fully incomplete runs. These results provide a defensible midterm answer to the first project objective and identify the gateway as the leading bottleneck candidate for the final report, pending confirmation from service-level observability data.


## I. Introduction

P.I.T.C.H. is a cloud-native microservice system composed of a gateway, multiple NestJS backend services, asynchronous messaging over RabbitMQ, Redis caching, persistent databases, and a Next.js frontend. The goal of this project is to evaluate how the system behaves as concurrent demand increases, determine where bottlenecks emerge, and quantify the impact of future scaling decisions.

## II. Background and Motivation

Microservice architectures offer deployment flexibility and independent scaling, but they also introduce network, queuing, coordination, and observability complexity. For P.I.T.C.H., performance matters directly to user experience because requests traverse a gateway, backend services, and messaging infrastructure. The motivation for this study is to move from architecture assumptions to measured evidence and use that evidence to identify the system limits and the most valuable optimization targets.

## III. Performance Evaluation Objectives

1. Measure how response time and throughput change as concurrent load increases.

2. Identify which service or dependency becomes the bottleneck under stress.

3. Quantify how scaling and caching affect latency, throughput, and stability in later experiments.

## IV. Evaluation Technique

The current evaluation uses quantitative, measurement-based testing. Closed-workload load generation is performed with k6, and summary metrics are exported to JSON after each run. The measurements emphasized in the current dataset are mean latency, P95 latency, P99 latency, throughput, and error rate. Warm-up, steady-state, and cool-down phases are included in each run to reduce startup bias.

## V. Evaluation Setup

The system under test is deployed in Docker containers. Current collected runs target the gateway entry point and exercise the light API path through the microservice stack. The broader observability environment includes Prometheus, Grafana, RabbitMQ, Redis, container monitoring, and application metrics, though this report focuses on the k6 results collected so far.

## VI. Test Workloads

The current midterm dataset includes baseline characterization of the light endpoint and light API load scaling at 10, 50, 100, 200, and 500 virtual users. AI-intensive, mixed, spike, stress, replica-comparison, and cache-comparison workloads are not yet included in this report and remain part of the final-evaluation plan.

## VII. Experimental Design

For the light workload, the target concurrency levels were 10, 50, 100, 200, and 500 virtual users. Five repeated runs were planned per load level. At the highest load levels, threshold failures and gateway instability emerged, so some runs represent overload behavior rather than clean steady-state service. In particular, some of the 500-virtual-user runs became incomplete and must be treated separately during analysis.

## VIII. Data Collection

Data was collected from the k6 summary JSON files stored under perf/results. Two valid baseline runs were available after excluding one incomplete failed baseline. For load scaling, the light workload dataset now includes 5 runs at 10 VUs, 5 runs at 50 VUs, 4 runs at 100 VUs, 5 runs at 200 VUs, and 5 runs at 500 VUs. Of the 500-VU runs, only 2 are valid complete runs; the remaining 3 are incomplete overload runs with zero-byte transfers and should not be averaged with the valid runs.

Host machine used for data collection:

- Model: MacBook Pro (Mac16,8)
- Processor: Apple M4 Pro
- CPU cores: 12 (8 performance and 4 efficiency)
- Memory: 24 GB
- Operating system: macOS 26.2 (25C56)
- Kernel: Darwin 25.2.0

### VIII.A. Data Validity Note

An important threat to validity is that the data was collected on a personal laptop that was also being used for normal student work during the testing window. Background applications, browser activity, course-related development tools, and other foreground tasks were not fully eliminated. As a result, the measured values should be interpreted as realistic development-machine results rather than fully isolated laboratory measurements. The concurrency trends and failure boundaries are still useful, but the absolute latency and throughput numbers may include noise from unrelated host activity.

## IX. Initial Results

Baseline performance was strong, with mean latency 3.33 ms, P95 latency 7.81 ms, and throughput 13.44 req/s. At 10 VUs the system remained effectively identical to baseline.

By 50 VUs, throughput increased to 48.95 req/s, but P99 latency increased to 2589.82 ms. At 100 VUs, throughput increased further to 87.70 req/s while P95 reached 844.77 ms and P99 reached 3999.72 ms. These results indicate that tail latency degradation begins well before visible request failures.

At 200 VUs, the light endpoint entered an instability regime. Across 5 runs, the aggregate error rate reached 13.79%, mean latency reached 6624.51 ms, and P99 latency reached 310982.23 ms. One run exhibited catastrophic behavior with P99 latency above 1524.17 seconds.

At 500 VUs, overload behavior became explicit. The first valid 500-VU run still delivered 333.12 req/s, but with P95 latency 4040.54 ms. The second valid run degraded to an error rate of 43.19% and a P99 of 66832.95 ms. The remaining 3 runs were incomplete collapse runs.

The current evidence supports the following midterm conclusion: the light API path is healthy at low concurrency, reaches a clear tail-latency cliff between 10 and 50 virtual users, becomes saturated but still functional at 100 virtual users, turns unstable at 200 virtual users, and can collapse at 500 virtual users. Based on observed connection-refused failures and previous gateway restarts during the high-load runs, the API gateway is the leading bottleneck candidate, although this still needs to be confirmed using Prometheus and Grafana correlation in the final report.

| Scenario | Valid runs | Mean latency (ms) | P95 latency (ms) | P99 latency (ms) | Throughput (req/s) | Error rate (%) |
|---|---|---|---|---|---|---|
| light / 10 VUs | 5 | 3.41 | 8.35 | 16.90 | 13.43 | 0.00 |
| light / 50 VUs | 5 | 82.85 | 229.84 | 2589.82 | 48.95 | 0.00 |
| light / 100 VUs | 4 | 156.58 | 844.77 | 3999.72 | 87.70 | 0.00 |
| light / 200 VUs | 5 | 6624.51 | 2390.95 | 310982.23 | 102.40 | 13.79 |
| light / 500 VUs | 2 | 1060.99 | 4290.31 | 37316.64 | 237.71 | 21.96 |

*Table I. Aggregate light-workload results across concurrent user levels.*

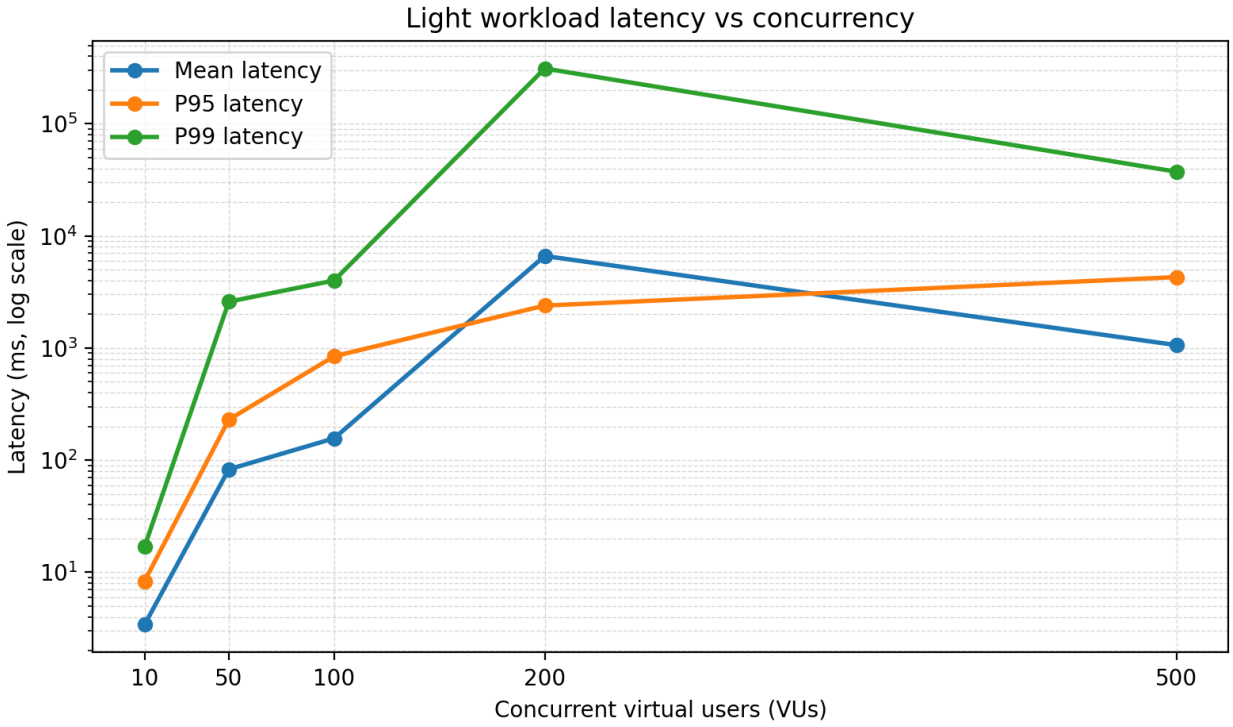


*Fig. 1. Light workload latency vs concurrency.*

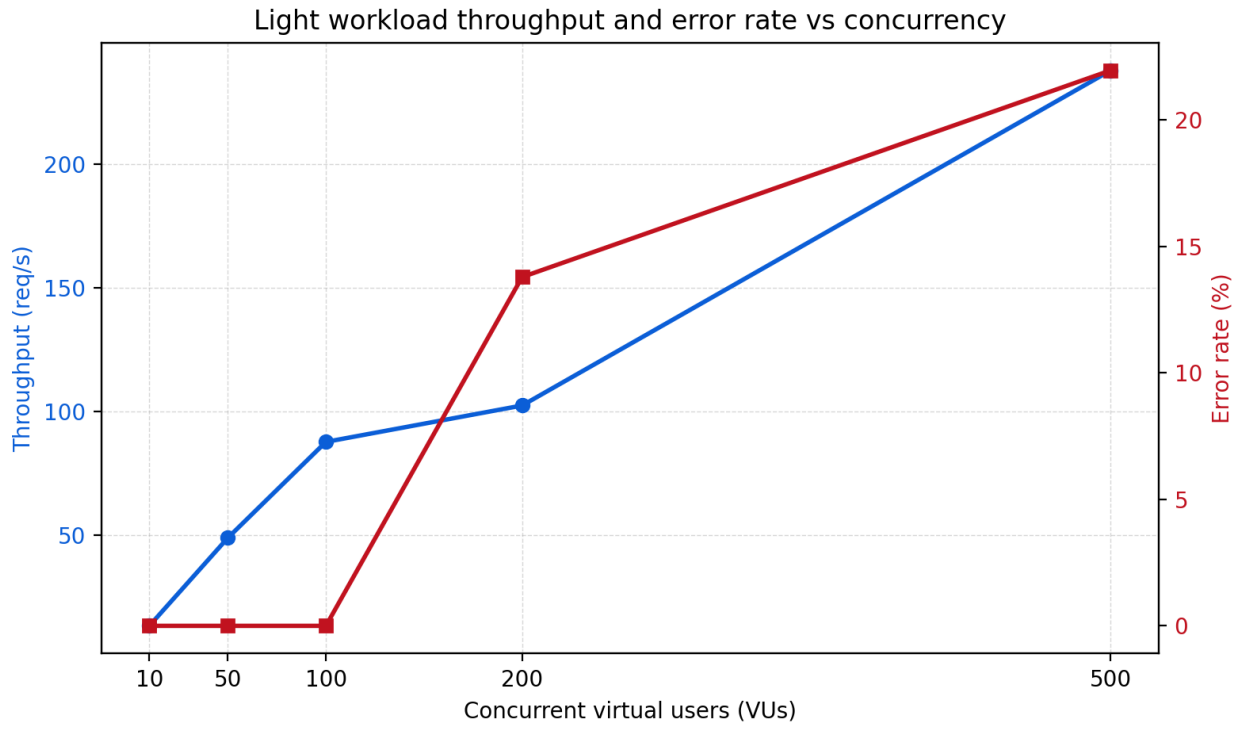


*Fig. 2. Light workload throughput and error rate vs concurrency.*

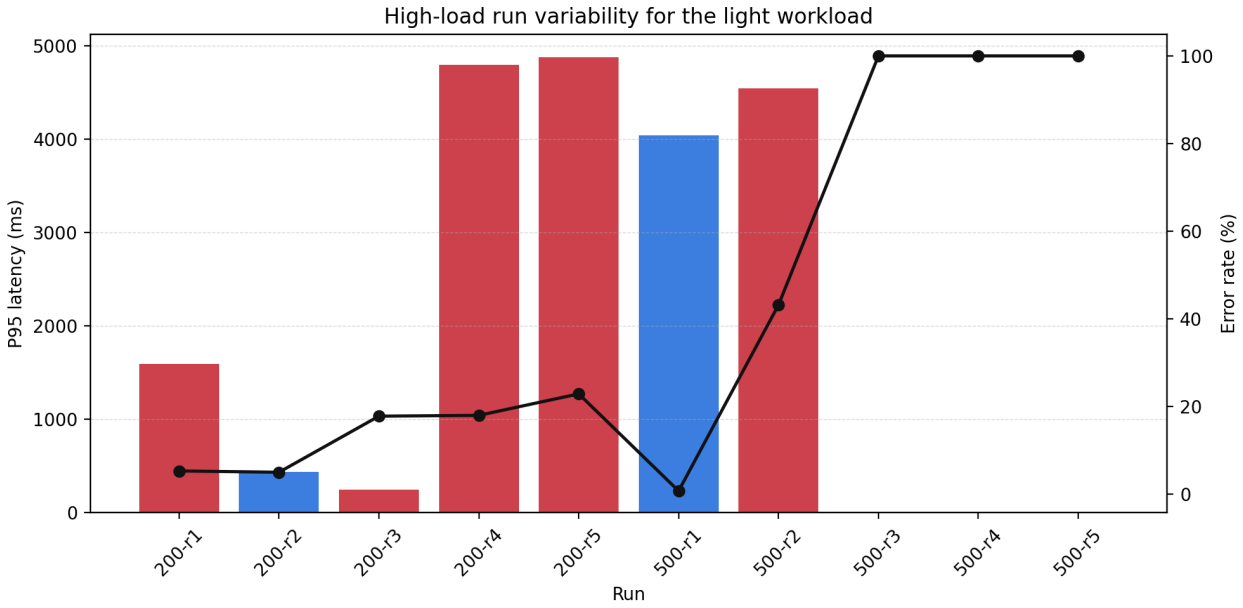


*Fig. 3. High-load run variability for the light workload.*

| High-load run | Incomplete | Mean latency (ms) | P95 latency (ms) | P99 latency (ms) | Throughput (req/s) | Error rate (%) |
|---|---|---|---|---|---|---|
| 200 / run-1.json | no | 442.23 | 1593.90 | 9686.62 | 111.51 | 5.30 |
| 200 / run-2.json | no | 361.45 | 437.10 | 6174.20 | 148.89 | 5.00 |
| 200 / run- | no | 220.00 | 250.26 | 4828.47 | 159.06 | 17.80 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 3.json | | | | | | |
| 200 / run-4.json | no | 31302.96 | 4795.66 | 1524166.44 | 2.34 | 17.98 |
| 200 / run-5.json | no | 795.91 | 4877.86 | 10055.43 | 90.17 | 22.88 |
| 500 / run-1.json | no | 450.54 | 4040.54 | 7800.33 | 333.12 | 0.73 |
| 500 / run-2.json | no | 1671.44 | 4540.08 | 66832.95 | 142.30 | 43.19 |
| 500 / run-3.json | yes | 0.00 | 0.00 | 0.00 | 108.94 | 100.00 |
| 500 / run-4.json | yes | 0.00 | 0.00 | 0.00 | 490.96 | 100.00 |
| 500 / run-5.json | yes | 0.00 | 0.00 | 0.00 | 746.20 | 100.00 |

*Table II. Per-run high-load results for 200 and 500 virtual users.*

## Appendix A. Changes to Project Proposal

The main change since the proposal is that the midterm dataset currently covers only the baseline and light API workload. The AI-intensive, mixed, spike, stress, scaling, and cache-comparison stages remain planned for the final report. In addition, high-load threshold failures required updates to the test harness so that failed runs could be preserved and the remaining batch could continue.

## Appendix B. Updated Milestones

| Date | Milestone |
|---|---|
| March 22, 2026 | Midterm report with baseline and light load-scaling results |
| March 26, 2026 | Complete AI and mixed workload load-scaling experiments |
| March 30, 2026 | Correlate high-load failures with Prometheus and Grafana metrics |
| April 5, 2026 | Complete scaling, cache, and bottleneck-validation experiments |
| April 10, 2026 | Final report submission |